MANUAL DEL PROGRAMADOR

Piedra, Papel o Tijera

Actividad 7 – Persistencia de datos con struct + fstream

Proyecto	piedrapapelotijera
Actividad	Actividad 7 – LPR 2026
Versión del documento	v1.0.0
Fecha	04/11/2026
Integrantes	Ian Quiroga, Santino Laudati y Dante Iglesias
Institución	E.E.S.T. N.° 1 "Eduardo Ader" – Vicente López
Materia / Curso	Laboratorio de Programación (LPR) – 5° 3°
Docente	Prof. Mansilla Muñoz York E.

Control de revisiones (DocVer)

Versión	Fecha	Descripción	Autores
v1.0.0	04/11/2026	Baseline Aprobado	I. Quiroga, S. Laudati, D. Iglesias

1. Propósito del documento

Este manual técnico documenta la arquitectura del mini juego **Piedra, Papel o Tijera** con persistencia en disco: el modelo de datos, las funciones, el formato del archivo, la compilación y las pautas para extenderlo o adaptarlo al Proyecto Integrador Anual.

2. Requisitos

Herramienta	Versión
g++ (MSYS2 UCRT64)	16.1.0 (compilado con `-std=c++11`)
Bibliotecas estándar	`<iostream>`, `<fstream>`, `<string>`, `<limits>`, `<cstdlib>`, `<ctime>`
Git	2.x – Dual-Remote GitHub + GitLab

3. Estructura del repositorio

```
piedrapapelotijera/
├── .gitattributes  ├── .gitignore  ├── LICENSE  ├── README.md
├── docs/
│   ├── InformeEEST1_LPR2026_ACT07_G03_QuirogaLaudatiIglesias_Informe_v1.0.0.pdf
│   ├── CHANGELOG.md
│   └── manuales/ manual_programador_v1.0.0.(md|pdf), manual_usuario_v1.0.0.(md|pdf)
├── src/main.cpp
├── data/historial_partidas.txt      # archivo persistente (CSV)
└── capturas/
```

4. Modelo de datos

```
struct Partida {
    int    idPartida;        // numero correlativo
    string jugador;          // nombre o nickname
    string eleccionJugador;  // "Piedra" | "Papel" | "Tijera"
    string eleccionIA;       // eleccion aleatoria de la IA
    string resultado;        // "Victoria" | "Derrota" | "Empate"
};
const string RUTA_HISTORIAL = "data/historial_partidas.txt";
```

5. Referencia de funciones

Función	Parámetros	Descripción
`int main()`	—	Inicializa la semilla aleatoria, calcula el próximo ID y muestra el menú en un ciclo `do-while` hasta elegir 3.
`void jugarPartida(int& contadorID)`	Contador por referencia	Pide nombre y jugada (validada 1–3), sortea la jugada de la IA, evalúa las reglas y llama a `guardarEnDisco()`. Incrementa el contador original.
`void guardarEnDisco(const Partida& p)`	Partida por referencia constante	Abre `ofstream` en modo `ios::app` y agrega una línea CSV.
`void leerHistorialDisco()`	—	Abre `ifstream`, lee cada línea separando por comas y muestra el total.
`int contarPartidasEnDisco()`	—	Cuenta las líneas no vacías del archivo para continuar la numeración.
`void limpiarEntrada()`	—	`cin.clear()` + `cin.ignore(...)` para recuperar `cin` tras un error.

5.1 Reglas del juego

Jugador	Le gana a
Piedra	Tijera
Papel	Piedra
Tijera	Papel

Si ambas elecciones son iguales el resultado es **Empate**.

6. Formato del archivo persistente

Una partida por línea, campos separados por comas (CSV, sin encabezado):

```
id,jugador,eleccionJugador,eleccionIA,resultado
1,Ian Quiroga,Piedra,Tijera,Victoria
```

> El nombre del jugador no debe contener comas, porque la coma es el separador de campos.

7. Compilación y ejecución

```
g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe
.\src\piedrapapelotijera.exe
```

Ejecutar siempre desde la raíz del proyecto: la ruta `data/historial_partidas.txt` es relativa al directorio de trabajo.

8. Pruebas realizadas

Caso	Resultado
Opción 2 sin archivo	"[AVISO] Aun no existen partidas guardadas"
Jugada fuera de rango (9)	Mensaje de error y se repite la pregunta
Letras en el menú	"[ERROR] Opcion invalida" sin bucle infinito
Cerrar y reabrir la consola	Historial recuperado; IDs continúan

9. Adaptación al P.I.A.

Reemplazar los campos de `Partida` por los del evento a registrar (por ejemplo `idLectura`, `sensor`, `valor`, `estado`), mantener `ios::app` para no perder registros y cambiar la extensión a `.csv` si se desea abrir el archivo en una planilla de cálculo.